

REGISTRATION NUMBER – 20233275

NAME – SOUVIK SARKAR

SECTION – D1

ASSIGNMENT NUMBER – 9

1)

CODE

```
import javax.swing.*;
import javax.swing.table.DefaultTableModel;
import java.awt.*;
import java.awt.event.*;
import java.sql.*;

public class StudentDetailsJDBC {
    private static final String DB_URL = "jdbc:mysql
    private static final String USER = "your_username";
    private static final String PASS = "your_password";

    private JFrame frame;
    private JTextField nameField, ageField, rollField, gradeField;
    private JTable table;
    private DefaultTableModel tableModel;

    public StudentDetailsJDBC() {
        frame = new JFrame("Student Details Form");
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.setSize(600, 500);
        frame.setLayout(new BorderLayout());
```

```
JPanel inputPanel = new JPanel(new GridLayout(5, 2, 10, 10));  
inputPanel.setBorder(BorderFactory.createEmptyBorder(10, 10, 10, 10));
```

```
inputPanel.add(new JLabel("Name:"));  
nameField = new JTextField();  
inputPanel.add(nameField);
```

```
inputPanel.add(new JLabel("Age:"));  
ageField = new JTextField();  
inputPanel.add(ageField);
```

```
inputPanel.add(new JLabel("Roll Number:"));  
rollField = new JTextField();  
inputPanel.add(rollField);
```

```
inputPanel.add(new JLabel("Grade:"));  
gradeField = new JTextField();  
inputPanel.add(gradeField);
```

```
JButton submitButton = new JButton("Submit");  
inputPanel.add(submitButton);
```

```
JButton clearButton = new JButton("Clear");  
inputPanel.add(clearButton);
```

```
frame.add(inputPanel, BorderLayout.NORTH);
```

```
tableModel = new DefaultTableModel(new String[]{"ID", "Name", "Age", "Roll Number",  
"Grade"}, 0);
```

```
table = new JTable(tableModel);
```

```
JScrollPane scrollPane = new JScrollPane(table);
```

```
frame.add(scrollPane, BorderLayout.CENTER);
```

```
fetchAndDisplayStudents();
```

```
submitButton.addActionListener(e -> insertStudent());
```

```
clearButton.addActionListener(e -> clearForm());
```

```
frame.setVisible(true);
```

```
}
```

```
private void clearForm() {
```

```
    nameField.setText("");
```

```
    ageField.setText("");
```

```
    rollField.setText("");
```

```
    gradeField.setText("");
```

```
}
```

```
private void insertStudent() {
```

```
    String name = nameField.getText().trim();
```

```
    String ageText = ageField.getText().trim();
```

```
    String rollNumber = rollField.getText().trim();
```

```
    String grade = gradeField.getText().trim();
```

```

        if (name.isEmpty() || ageText.isEmpty() || rollNumber.isEmpty() || grade.isEmpty()) {
            JOptionPane.showMessageDialog(frame, "Please fill all fields.", "Input Error",
JOptionPane.ERROR_MESSAGE);
            return;
        }

        int age;
        try {
            age = Integer.parseInt(ageText);
            if(age <= 0){
                JOptionPane.showMessageDialog(frame, "Age must be positive.", "Input Error",
JOptionPane.ERROR_MESSAGE);
                return;
            }
        } catch (NumberFormatException ex) {
            JOptionPane.showMessageDialog(frame, "Please enter a valid age.", "Input Error",
JOptionPane.ERROR_MESSAGE);
            return;
        }

        String insertSQL = "INSERT INTO students (name, age, roll_number, grade) VALUES (?, ?,
?, ?)";

        try (Connection conn = DriverManager.getConnection(DB_URL, USER, PASS);
            PreparedStatement pstmt = conn.prepareStatement(insertSQL)) {

            pstmt.setString(1, name);
            pstmt.setInt(2, age);
            pstmt.setString(3, rollNumber);
            pstmt.setString(4, grade);

```

```

        int rowsInserted = pstmt.executeUpdate();
        if (rowsInserted > 0) {
            JOptionPane.showMessageDialog(frame, "Student details added successfully!",
"Success", JOptionPane.INFORMATION_MESSAGE);
            clearForm();
            fetchAndDisplayStudents();
        }

    } catch (SQLException ex) {
        ex.printStackTrace();
        JOptionPane.showMessageDialog(frame, "Error inserting data: " + ex.getMessage(),
"Database Error", JOptionPane.ERROR_MESSAGE);
    }
}

```

```

private void fetchAndDisplayStudents() {
    String selectSQL = "SELECT * FROM students";

    tableModel.setRowCount(0);

    try (Connection conn = DriverManager.getConnection(DB_URL, USER, PASS);
        Statement stmt = conn.createStatement();
        ResultSet rs = stmt.executeQuery(selectSQL)) {

        while (rs.next()) {
            int id = rs.getInt("id");
            String name = rs.getString("name");
            int age = rs.getInt("age");
            String rollNumber = rs.getString("roll_number");
            String grade = rs.getString("grade");

```

```

        tableModel.addRow(new Object[]{id, name, age, rollNumber, grade});
    }

} catch (SQLException ex) {
    ex.printStackTrace();

    JOptionPane.showMessageDialog(frame, "Error fetching data: " + ex.getMessage(),
    "Database Error", JOptionPane.ERROR_MESSAGE);
}
}

public static void main(String[] args) {
    try {
        Class.forName("com.mysql.cj.jdbc.Driver");
    } catch (ClassNotFoundException e) {
        e.printStackTrace();

        JOptionPane.showMessageDialog(null, "MySQL JDBC Driver not found. Please add it
        to the classpath.", "Driver Error", JOptionPane.ERROR_MESSAGE);

        return;
    }

    SwingUtilities.invokeLater(StudentDetailsJDBC::new);
}
}

```

2)

CODE

StudentDAO.java

```
import java.sql.*;
import java.util.ArrayList;
import java.util.List;
import java.util.Properties;
import java.io.InputStream;

public class StudentDAO {
    private Connection connect() throws Exception {
        Properties props = new Properties();

        try (InputStream input =
getClass().getClassLoader().getResourceAsStream("db.properties")) {
            props.load(input);
        }

        String url = props.getProperty("db.url");
        String user = props.getProperty("db.username");
        String password = props.getProperty("db.password");

        return DriverManager.getConnection(url, user, password);
    }

    public List<Student> getAllStudents() throws Exception {
```

```
List<Student> students = new ArrayList<>();
```

```
String query = "SELECT * FROM students";
```

```
try (Connection conn = connect();
```

```
    Statement stmt = conn.createStatement();
```

```
    ResultSet rs = stmt.executeQuery(query)) {
```

```
    while (rs.next()) {
```

```
        students.add(new Student(rs.getInt("id"), rs.getString("name"), rs.getInt("age"),  
                                rs.getString("roll_number"), rs.getString("grade")));
```

```
    }
```

```
}
```

```
return students;
```

```
}
```

```
public void addStudent(Student student) throws Exception {
```

```
    String query = "INSERT INTO students (name, age, roll_number, grade) VALUES (?, ?, ?,  
    ?)";
```

```
try (Connection conn = connect();
```

```
    PreparedStatement pstmt = conn.prepareStatement(query)) {
```

```
    pstmt.setString(1, student.getName());
```

```
    pstmt.setInt(2, student.getAge());
```

```
    pstmt.setString(3, student.getRollNumber());
```

```
    pstmt.setString(4, student.getGrade());
```

```
    pstmt.executeUpdate();
```

```
}
```

```
}
```

```
public void updateStudent(Student student) throws Exception {
```

```
String query = "UPDATE students SET name=?, age=?, roll_number=?, grade=? WHERE id=?";
```

```
try (Connection conn = connect();
    PreparedStatement pstmt = conn.prepareStatement(query)) {
    pstmt.setString(1, student.getName());
    pstmt.setInt(2, student.getAge());
    pstmt.setString(3, student.getRollNumber());
    pstmt.setString(4, student.getGrade());
    pstmt.setInt(5, student.getId());
    pstmt.executeUpdate();
}
}
```

```
public void deleteStudent(int id) throws Exception {
    String query = "DELETE FROM students WHERE id=?";
```

```
try (Connection conn = connect();
    PreparedStatement pstmt = conn.prepareStatement(query)) {
    pstmt.setInt(1, id);
    pstmt.executeUpdate();
}
}
}
```

Student.java

```
public class Student {
    private int id;
    private String name;
    private int age;
    private String rollNumber;
```

```
private String grade;
```

```
public Student(int id, String name, int age, String rollNumber, String grade) {
```

```
    this.id = id;
```

```
    this.name = name;
```

```
    this.age = age;
```

```
    this.rollNumber = rollNumber;
```

```
    this.grade = grade;
```

```
}
```

```
}
```

3)

CODE

StudentDAO.java

```
import java.sql.*;
```

```
import java.util.ArrayList;
```

```
import java.util.List;
```

```
import java.util.Properties;
```

```
import java.io.InputStream;
```

```

public class StudentDAO {

    private Connection connect() throws Exception {

        Properties props = new Properties();

        try (InputStream input =
getClass().getClassLoader().getResourceAsStream("db.properties")) {
            props.load(input);
        }

        String url = props.getProperty("db.url");
        String user = props.getProperty("db.username");
        String password = props.getProperty("db.password");

        return DriverManager.getConnection(url, user, password);
    }

    public List<Student> getAllStudents() throws Exception {

        List<Student> students = new ArrayList<>();

        String query = "SELECT * FROM students";

        try (Connection conn = connect();
            Statement stmt = conn.createStatement();
            ResultSet rs = stmt.executeQuery(query)) {

            while (rs.next()) {

                students.add(new Student(rs.getInt("id"), rs.getString("name"), rs.getInt("age"),
                    rs.getString("roll_number"), rs.getString("grade")));
            }
        }

        return students;
    }
}

```

```

public void addStudent(Student student) throws Exception {
    String query = "INSERT INTO students (name, age, roll_number, grade) VALUES (?, ?, ?, ?)";

    try (Connection conn = connect();
        PreparedStatement pstmt = conn.prepareStatement(query)) {
        pstmt.setString(1, student.getName());
        pstmt.setInt(2, student.getAge());
        pstmt.setString(3, student.getRollNumber());
        pstmt.setString(4, student.getGrade());
        pstmt.executeUpdate();
    }
}

```

```

public void updateStudent(Student student) throws Exception {
    String query = "UPDATE students SET name=?, age=?, roll_number=?, grade=? WHERE id=?";

    try (Connection conn = connect();
        PreparedStatement pstmt = conn.prepareStatement(query)) {
        pstmt.setString(1, student.getName());
        pstmt.setInt(2, student.getAge());
        pstmt.setString(3, student.getRollNumber());
        pstmt.setString(4, student.getGrade());
        pstmt.setInt(5, student.getId());
        pstmt.executeUpdate();
    }
}

```

```

public void deleteStudent(int id) throws Exception {
    String query = "DELETE FROM students WHERE id=?";

    try (Connection conn = connect();
        PreparedStatement pstmt = conn.prepareStatement(query)) {
        pstmt.setInt(1, id);
        pstmt.executeUpdate();
    }
}

public List<Student> searchStudent(String searchTerm) throws Exception {
    List<Student> students = new ArrayList<>();
    String query = "SELECT * FROM students WHERE roll_number = ? OR name LIKE ?";

    try (Connection conn = connect();
        PreparedStatement pstmt = conn.prepareStatement(query)) {

        pstmt.setString(1, searchTerm);
        pstmt.setString(2, "%" + searchTerm + "%");

        try (ResultSet rs = pstmt.executeQuery()) {
            while (rs.next()) {
                students.add(new Student(rs.getInt("id"), rs.getString("name"), rs.getInt("age"),
                    rs.getString("roll_number"), rs.getString("grade")));
            }
        }
    }

    return students;
}

```

```
}
```

4)

CODE

StudentDAO.java

```
import java.sql.*;
```

```
import java.util.ArrayList;
```

```
import java.util.List;
```

```
public class StudentDAO {
```

```

private Connection connect() throws SQLException {
    String url = "jdbc:mysql://localhost:3306/StudentDB";
    String username = "your_db_username";
    String password = "your_db_password";
    return DriverManager.getConnection(url, username, password);
}

```

```

public void addStudent(Student student) throws SQLException {
    String query = "INSERT INTO students (name, age, roll_number, grade) VALUES (?, ?, ?, ?)";
    try (Connection conn = connect(); PreparedStatement pstmt =
conn.prepareStatement(query)) {
        pstmt.setString(1, student.getName());
        pstmt.setInt(2, student.getAge());
        pstmt.setString(3, student.getRollNumber());
        pstmt.setString(4, student.getGrade());
        pstmt.executeUpdate();
    }
}

```

```

public List<Student> getAllStudents() throws SQLException {
    List<Student> students = new ArrayList<>();
    String query = "SELECT * FROM students";
    try (Connection conn = connect(); Statement stmt = conn.createStatement(); ResultSet
rs = stmt.executeQuery(query)) {
        while (rs.next()) {
            students.add(new Student(rs.getInt("id"), rs.getString("name"), rs.getInt("age"),
rs.getString("roll_number"), rs.getString("grade")));
        }
    }
    return students;
}

```

```
}
```

```
public void updateStudent(Student student) throws SQLException {
```

```
    String query = "UPDATE students SET name=?, age=?, grade=? WHERE roll_number=?";
```

```
    try (Connection conn = connect(); PreparedStatement pstmt =  
conn.prepareStatement(query)) {
```

```
        pstmt.setString(1, student.getName());
```

```
        pstmt.setInt(2, student.getAge());
```

```
        pstmt.setString(3, student.getGrade());
```

```
        pstmt.setString(4, student.getRollNumber());
```

```
        pstmt.executeUpdate();
```

```
    }
```

```
}
```

```
public void deleteStudent(String rollNumber) throws SQLException {
```

```
    String query = "DELETE FROM students WHERE roll_number=?";
```

```
    try (Connection conn = connect(); PreparedStatement pstmt =  
conn.prepareStatement(query)) {
```

```
        pstmt.setString(1, rollNumber);
```

```
        pstmt.executeUpdate();
```

```
    }
```

```
}
```

```
public List<Student> searchStudent(String searchTerm) throws SQLException {
```

```
    List<Student> students = new ArrayList<>();
```

```
    String query = "SELECT * FROM students WHERE roll_number = ? OR name LIKE ?";
```

```
    try (Connection conn = connect(); PreparedStatement pstmt =  
conn.prepareStatement(query)) {
```

```
        pstmt.setString(1, searchTerm);
```

```
        pstmt.setString(2, "%" + searchTerm + "%");
```

```

        try (ResultSet rs = pstmt.executeQuery()) {
            while (rs.next()) {
                students.add(new Student(rs.getInt("id"), rs.getString("name"), rs.getInt("age"),
rs.getString("roll_number"), rs.getString("grade")));
            }
        }
    }
    return students;
}
}

```

Student.java

```

import javax.swing.*.*;
import javax.swing.table.DefaultTableModel;
import java.awt.event.*;
import java.sql.SQLException;
import java.util.List;

public class StudentManagementApp {
    private JFrame frame;
    private JTextField txtName, txtAge, txtRollNumber, txtGrade, txtSearch;
    private JTable table;
    private DefaultTableModel model;
    private StudentDAO studentDAO;

    public StudentManagementApp() {
        studentDAO = new StudentDAO();
        frame = new JFrame("Student Management System");
        frame.setSize(600, 400);
    }
}

```

```
frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);  
frame.setLayout(null);
```

```
JLabel lblName = new JLabel("Name:");  
lblName.setBounds(20, 20, 100, 25);  
frame.add(lblName);
```

```
txtName = new JTextField();  
txtName.setBounds(120, 20, 150, 25);  
frame.add(txtName);
```

```
JLabel lblAge = new JLabel("Age:");  
lblAge.setBounds(20, 60, 100, 25);  
frame.add(lblAge);
```

```
txtAge = new JTextField();  
txtAge.setBounds(120, 60, 150, 25);  
frame.add(txtAge);
```

```
JLabel lblRollNumber = new JLabel("Roll Number:");  
lblRollNumber.setBounds(20, 100, 100, 25);  
frame.add(lblRollNumber);
```

```
txtRollNumber = new JTextField();  
txtRollNumber.setBounds(120, 100, 150, 25);  
frame.add(txtRollNumber);
```

```
JLabel lblGrade = new JLabel("Grade:");  
lblGrade.setBounds(20, 140, 100, 25);  
frame.add(lblGrade);
```

```
txtGrade = new JTextField();  
txtGrade.setBounds(120, 140, 150, 25);  
frame.add(txtGrade);
```

```
JButton btnAdd = new JButton("Add Student");  
btnAdd.setBounds(300, 20, 150, 25);  
frame.add(btnAdd);
```

```
JButton btnUpdate = new JButton("Update Student");  
btnUpdate.setBounds(300, 60, 150, 25);  
frame.add(btnUpdate);
```

```
JButton btnDelete = new JButton("Delete Student");  
btnDelete.setBounds(300, 100, 150, 25);  
frame.add(btnDelete);
```

```
JButton btnViewAll = new JButton("View All Students");  
btnViewAll.setBounds(300, 140, 150, 25);  
frame.add(btnViewAll);
```

```
JLabel lblSearch = new JLabel("Search (Roll No/Name):");
```

```
lblSearch.setBounds(20, 180, 150, 25);
```

```
frame.add(lblSearch);
```

```
txtSearch = new JTextField();
```

```
txtSearch.setBounds(180, 180, 150, 25);
```

```
frame.add(txtSearch);
```

```
JButton btnSearch = new JButton("Search");
```

```
btnSearch.setBounds(340, 180, 100, 25);
```

```
frame.add(btnSearch);
```

```
model = new DefaultTableModel();
```

```
table = new JTable(model);
```

```
model.addColumn("ID");
```

```
model.addColumn("Name");
```

```
model.addColumn("Age");
```

```
model.addColumn("Roll Number");
```

```
model.addColumn("Grade");
```

```
JScrollPane scrollPane = new JScrollPane(table);
```

```
scrollPane.setBounds(20, 220, 540, 130);
```

```
frame.add(scrollPane);
```

```
// Action listeners
```

```
btnAdd.addActionListener(e -> addStudent());
```

```
btnUpdate.addActionListener(e -> updateStudent());
```

```

        btnDelete.addActionListener(e -> deleteStudent());
        btnViewAll.addActionListener(e -> viewAllStudents());
        btnSearch.addActionListener(e -> searchStudent());

        frame.setVisible(true);
    }

    private void addStudent() {
        try {
            Student student = new Student(0, txtName.getText(),
            Integer.parseInt(txtAge.getText()), txtRollNumber.getText(), txtGrade.getText());
            studentDAO.addStudent(student);
            JOptionPane.showMessageDialog(frame, "Student added successfully.");
            clearFields();
        } catch (SQLException e) {
            e.printStackTrace();
            JOptionPane.showMessageDialog(frame, "Error adding student.");
        }
    }

    private void updateStudent() {
        try {
            Student student = new Student(0, txtName.getText(),
            Integer.parseInt(txtAge.getText()), txtRollNumber.getText(), txtGrade.getText());
            studentDAO.updateStudent(student);
            JOptionPane.showMessageDialog(frame, "Student updated
            successfully.");
        }
    }

```

```
        clearFields();
    } catch (SQLException e) {
        e.printStackTrace();
        JOptionPane.showMessageDialog(frame, "Error updating student.");
    }
}
```

```
private void deleteStudent() {
    try {
        String rollNumber = txtRollNumber.getText();
        studentDAO.deleteStudent(rollNumber);
        JOptionPane.showMessageDialog(frame, "Student deleted
successfully.");
        clearFields();
    } catch (SQLException e) {
        e.printStackTrace();
        JOptionPane.showMessageDialog(frame, "Error deleting student.");
    }
}
```

```
private void viewAllStudents() {
    try {
        List<Student> students = studentDAO.getAllStudents();
        updateTable(students);
    } catch (SQLException e) {
        e.printStackTrace();
    }
}
```

```
}
```

```
private void searchStudent() {  
    try {  
        String searchTerm = txtSearch.getText();  
        List<Student> students = studentDAO.searchStudent(searchTerm);  
        updateTable(students);  
    } catch (SQLException e) {  
        e.printStackTrace();  
    }  
}
```

```
private void updateTable(List<Student> students) {  
    model.setRowCount(0);  
    for (Student student : students) {  
        model.addRow(new Object[]{student.getId(), student.getName(),  
student.getAge(), student.getRollNumber(), student.getGrade()});  
    }  
}
```

```
private void clearFields() {  
    txtName.setText("");  
    txtAge.setText("");  
    txtRollNumber.setText("");  
    txtGrade.setText("");  
    txtSearch.setText("");  
}
```

```
public static void main(String[] args) {  
    SwingUtilities.invokeLater(StudentManagementApp::new);  
}  
}
```